WIKIPEDIA 25
25 years of the free encyclopedia

Multilayer perceptron

In deep learning, a **multilayer perceptron** (**MLP**) is a kind of modern feedforward neural network consisting of fully connected neurons with nonlinear activation functions, organized in layers, notable for being able to distinguish data that is not linearly separable.[1]

Modern neural networks are trained using backpropagation[2][3][4][5][6] and are colloquially referred to as "vanilla" networks.[7] MLPs grew out of an effort to improve on single-layer perceptrons, which could only be applied to linearly separable data. A perceptron traditionally used a Heaviside step function as its nonlinear activation function. However, the backpropagation algorithm requires that modern MLPs use continuous activation functions such as sigmoid or ReLU.[8]

Multilayer perceptrons form the basis of deep learning,[9] and are applicable across a vast set of diverse domains.[10]

Timeline

- In 1943, Warren McCulloch and Walter Pitts proposed the binary artificial neuron as a logical model of biological neural networks.[11]
- In 1958, Frank Rosenblatt proposed the multilayered perceptron model, consisting of an input layer, a hidden layer with randomized weights that did not learn, and an output layer with learnable connections.[12]
- In 1962, Rosenblatt published many variants and experiments on perceptrons in his book *Principles of Neurodynamics*, including up to 2 trainable layers by "back-propagating errors".[13] However, it was not the backpropagation algorithm, and he did not have a general method for training multiple layers.
- In 1965, Alexey Grigorevich Ivakhnenko and Valentin Lapa published Group Method of Data Handling. It was one of the first deep learning methods, used to train an eight-layer neural net in 1971.[14][15][16]
- In 1967, Shun'ichi Amari reported [17] the first multilayered neural network trained by stochastic gradient descent, was able to classify non-linearily separable pattern classes. Amari's student Saito conducted the computer experiments, using a five-layered feedforward network with two learning layers.[16]
- Backpropagation was independently developed multiple times in early 1970s. The earliest published instance was Seppo Linnainmaa's master thesis (1970).[18][19][16] Paul Werbos developed it independently in 1971,[20] but had difficulty publishing it until 1982.[21]
- In 1986, David E. Rumelhart et al. popularized backpropagation.[22][23]
- In 2003, interest in backpropagation networks returned due to the successes of deep learning being applied to language modelling by Yoshua Bengio with co-authors.[24]
- In 2021, a very simple NN architecture combining two deep MLPs with skip connections and layer normalizations was designed and called MLP-Mixer; its realizations featuring 19 to 431

millions of parameters were shown to be comparable to vision transformers of similar size on ImageNet and similar image classification tasks.[25]

Mathematical foundations

Activation function

If a multilayer perceptron has a linear activation function in all neurons, that is, a linear function that maps the weighted inputs to the output of each neuron, then linear algebra shows that any number of layers can be reduced to a two-layer input-output model. In MLPs some neurons use a *nonlinear* activation function that was developed to model the frequency of action potentials, or firing, of biological neurons.

The two historically common activation functions are both sigmoids, and are described by

$$y(v_i) = \tanh(v_i) \text{ and } y(v_i) = (1 + e^{-v_i})^{-1}.$$

The first is a hyperbolic tangent that ranges from −1 to 1, while the other is the logistic function, which is similar in shape but ranges from 0 to 1. Here y_i is the output of the ith node (neuron) and v_i is the weighted sum of the input connections. Alternative activation functions have been proposed, including the rectifier and softplus functions. More specialized activation functions include radial basis functions (used in radial basis networks, another class of supervised neural network models).

In recent developments of deep learning the rectified linear unit (ReLU) is more frequently used as one of the possible ways to overcome the numerical problems related to the sigmoids.

Layers

The MLP consists of three or more layers (an input and an output layer with one or more *hidden layers*) of nonlinearly-activating nodes. Since MLPs are fully connected, each node in one layer connects with a certain weight w_{ij} to every node in the following layer.

Learning

Learning occurs in the perceptron by changing connection weights after each piece of data is processed, based on the amount of error in the output compared to the expected result. This is an example of supervised learning, and is carried out through backpropagation, a generalization of the least mean squares algorithm in the linear perceptron.

We can represent the degree of error in an output node j in the nth data point (training example) by $e_j(n) = d_j(n) - y_j(n)$, where $d_j(n)$ is the desired target value for nth data point at node j, and $y_j(n)$ is the value produced by the perceptron at node j when the nth data point is given as an input.

The node weights can then be adjusted based on corrections that minimize the error in the entire output for the nth data point, given by

$$\mathcal{E}(n) = \frac{1}{2} \sum_{\text{output node } j} e_j^2(n).$$

Using gradient descent, the change in each weight w_{ij} is

$$\Delta w_{ji}(n) = -\eta \frac{\partial \mathcal{E}(n)}{\partial v_j(n)} y_i(n)$$

where $y_i(n)$ is the output of the previous neuron i, and η is the *learning rate*, which is selected to ensure that the weights quickly converge to a response, without oscillations. In the previous expression, $\frac{\partial \mathcal{E}(n)}{\partial v_j(n)}$ denotes the partial derivate of the error $\mathcal{E}(n)$ according to the weighted sum $v_j(n)$ of the input connections of neuron i.

The derivative to be calculated depends on the induced local field v_j, which itself varies. It is easy to prove that for an output node this derivative can be simplified to

$$-\frac{\partial \mathcal{E}(n)}{\partial v_j(n)} = e_j(n)\phi'(v_j(n))$$

where ϕ' is the derivative of the activation function described above, which itself does not vary. The analysis is more difficult for the change in weights to a hidden node, but it can be shown that the relevant derivative is

$$-\frac{\partial \mathcal{E}(n)}{\partial v_j(n)} = \phi'(v_j(n)) \sum_k -\frac{\partial \mathcal{E}(n)}{\partial v_k(n)} w_{kj}(n).$$

This depends on the change in weights of the kth nodes, which represent the output layer. So to change the hidden layer weights, the output layer weights change according to the derivative of the activation function, and so this algorithm represents a backpropagation of the activation function.[26]

References

1. Cybenko, G. 1989. Approximation by superpositions of a sigmoidal function *Mathematics of Control, Signals, and Systems*, 2(4), 303–314.
2. Linnainmaa, Seppo (1970). *The representation of the cumulative rounding error of an algorithm as a Taylor expansion of the local rounding errors* (Masters) (in Finnish). University of Helsinki. pp. 6–7.
3. Kelley, Henry J. (1960). "Gradient theory of optimal flight paths". *ARS Journal*. **30** (10): 947–954. doi:10.2514/8.5282 (https://doi.org/10.2514%2F8.5282).
4. Rosenblatt, Frank. x. Principles of Neurodynamics: Perceptrons and the Theory of Brain Mechanisms. Spartan Books, Washington DC, 1961
5. Werbos, Paul (1982). "Applications of advances in nonlinear sensitivity analysis" (http://werbos.com/Neural/SensitivityIFIPSeptember1981.pdf) (PDF). *System modeling and optimization*. Springer. pp. 762–770. Archived (https://web.archive.org/web/20160414055503/http://werbos.com/Neural/SensitivityIFIPSeptember1981.pdf) (PDF) from the original on 14 April 2016. Retrieved 2 July 2017.

6. Rumelhart, David E., Geoffrey E. Hinton, and R. J. Williams. "Learning Internal Representations by Error Propagation (https://apps.dtic.mil/dtic/tr/fulltext/u2/a164453.pdf)". David E. Rumelhart, James L. McClelland, and the PDP research group. (editors), Parallel distributed processing: Explorations in the microstructure of cognition, Volume 1: Foundation. MIT Press, 1986.
7. Hastie, Trevor. Tibshirani, Robert. Friedman, Jerome. The Elements of Statistical Learning: Data Mining, Inference, and Prediction. Springer, New York, NY, 2009.
8. "Why is the ReLU function not differentiable at x=0?" (https://sebastianraschka.com/faq/docs/relu-derivative.html). 21 November 2024.
9. Almeida, Luis B (2020) [1996]. "Multilayer perceptrons" (https://www.taylorfrancis.com/chapters/edit/10.1201/9780429142772-60/multilayer-perceptrons-luis-almeida). In Fiesler, Emile; Beale, Russell (eds.). *Handbook of Neural Computation* (https://www.taylorfrancis.com/books/edit/10.1201/9780429142772/handbook-neural-computation-emile-fiesler-russell-beale?refId=fef0cae0-bd6d-4895-accc-744e0e281a0f&context=ubx). CRC Press. pp. C1-2. doi:10.1201/9780429142772 (https://doi.org/10.1201%2F9780429142772). ISBN 978-0-429-14277-2.
10. Gardner, Matt W; Dorling, Stephen R (1998). "Artificial neural networks (the multilayer perceptron)—a review of applications in the atmospheric sciences" (https://www.sciencedirect.com/science/article/abs/pii/S1352231097004470). *Atmospheric Environment*. **32** (14–15). Elsevier: 2627–2636. Bibcode:1998AtmEn..32.2627G (https://ui.adsabs.harvard.edu/abs/1998AtmEn..32.2627G). doi:10.1016/S1352-2310(97)00447-0 (https://doi.org/10.1016%2FS1352-2310%2897%2900447-0).
11. McCulloch, Warren S.; Pitts, Walter (1943-12-01). "A logical calculus of the ideas immanent in nervous activity" (https://doi.org/10.1007/BF02478259). *The Bulletin of Mathematical Biophysics*. **5** (4): 115–133. doi:10.1007/BF02478259 (https://doi.org/10.1007%2FBF02478259). ISSN 1522-9602 (https://search.worldcat.org/issn/1522-9602).
12. Rosenblatt, Frank (1958). "The Perceptron: A Probabilistic Model For Information Storage And Organization in the Brain". *Psychological Review*. **65** (6): 386–408. CiteSeerX 10.1.1.588.3775 (https://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.588.3775). doi:10.1037/h0042519 (https://doi.org/10.1037%2Fh0042519). PMID 13602029 (https://pubmed.ncbi.nlm.nih.gov/13602029). S2CID 12781225 (https://api.semanticscholar.org/CorpusID:12781225).
13. Rosenblatt, Frank (1962). *Principles of Neurodynamics*. Spartan, New York.
14. Ivakhnenko, A. G. (1973). *Cybernetic Predicting Devices* (https://books.google.com/books?id=FhwVNQAACAAJ). CCM Information Corporation.
15. Ivakhnenko, A. G.; Grigor′evich Lapa, Valentin (1967). *Cybernetics and forecasting techniques* (https://books.google.com/books?id=rGFgAAAAMAAJ). American Elsevier Pub. Co.
16. Schmidhuber, Juergen (2022). "Annotated History of Modern AI and Deep Learning". arXiv:2212.11279 (https://arxiv.org/abs/2212.11279) [cs.NE (https://arxiv.org/archive/cs.NE)].
17. Amari, Shun'ichi (1967). "A theory of adaptive pattern classifier". *IEEE Transactions*. **EC** (16): 279-307.
18. Linnainmaa, Seppo (1970). *The representation of the cumulative rounding error of an algorithm as a Taylor expansion of the local rounding errors* (Masters) (in Finnish). University of Helsinki. p. 6–7.
19. Linnainmaa, Seppo (1976). "Taylor expansion of the accumulated rounding error". *BIT Numerical Mathematics*. **16** (2): 146–160. doi:10.1007/bf01931367 (https://doi.org/10.1007%2Fbf01931367). S2CID 122357351 (https://api.semanticscholar.org/CorpusID:122357351).

20. Anderson, James A.; Rosenfeld, Edward, eds. (2000). *Talking Nets: An Oral History of Neural Networks* (https://direct.mit.edu/books/book/4886/Talking-NetsAn-Oral-History-of-Neural-Networks). The MIT Press. doi:10.7551/mitpress/6626.003.0016 (https://doi.org/10.7551%2Fmitpress%2F6626.003.0016). ISBN 978-0-262-26715-1.
21. Werbos, Paul (1982). "Applications of advances in nonlinear sensitivity analysis" (http://werbos.com/Neural/SensitivityIFIPSeptember1981.pdf) (PDF). *System modeling and optimization*. Springer. pp. 762–770. Archived (https://web.archive.org/web/20160414055503/http://werbos.com/Neural/SensitivityIFIPSeptember1981.pdf) (PDF) from the original on 14 April 2016. Retrieved 2 July 2017.
22. Rumelhart, David E.; Hinton, Geoffrey E.; Williams, Ronald J. (October 1986). "Learning representations by back-propagating errors" (https://www.nature.com/articles/323533a0). *Nature*. **323** (6088): 533–536. Bibcode:1986Natur.323..533R (https://ui.adsabs.harvard.edu/abs/1986Natur.323..533R). doi:10.1038/323533a0 (https://doi.org/10.1038%2F323533a0). ISSN 1476-4687 (https://search.worldcat.org/issn/1476-4687).
23. Rumelhart, David E., Geoffrey E. Hinton, and R. J. Williams. "Learning Internal Representations by Error Propagation (https://apps.dtic.mil/dtic/tr/fulltext/u2/a164453.pdf)". David E. Rumelhart, James L. McClelland, and the PDP research group. (editors), Parallel distributed processing: Explorations in the microstructure of cognition, Volume 1: Foundation. MIT Press, 1986.
24. Bengio, Yoshua; Ducharme, Réjean; Vincent, Pascal; Janvin, Christian (March 2003). "A neural probabilistic language model" (https://dl.acm.org/doi/10.5555/944919.944966). *The Journal of Machine Learning Research*. **3**: 1137–1155.
25. "Papers with Code – MLP-Mixer: An all-MLP Architecture for Vision" (https://paperswithcode.com/paper/mlp-mixer-an-all-mlp-architecture-for-vision).
26. Haykin, Simon (1998). *Neural Networks: A Comprehensive Foundation* (2 ed.). Prentice Hall. ISBN 0-13-273350-1.

External links

- Weka: Open source data mining software with multilayer perceptron implementation (https://ml.cms.waikato.ac.nz/weka/).
- Neuroph Studio documentation, implements this algorithm and a few others (https://neuroph.sourceforge.net/).

Retrieved from "https://en.wikipedia.org/w/index.php?title=Multilayer_perceptron&oldid=1316855641"